

Destination Atlas: Components working together

Destination Atlas provides information about thirty cities, five per inhabited continent, using every component in RosettaDash at least once. You browse places, look at trends, open a map, watch a video, and more. The app is a functional demo built around component workflows.

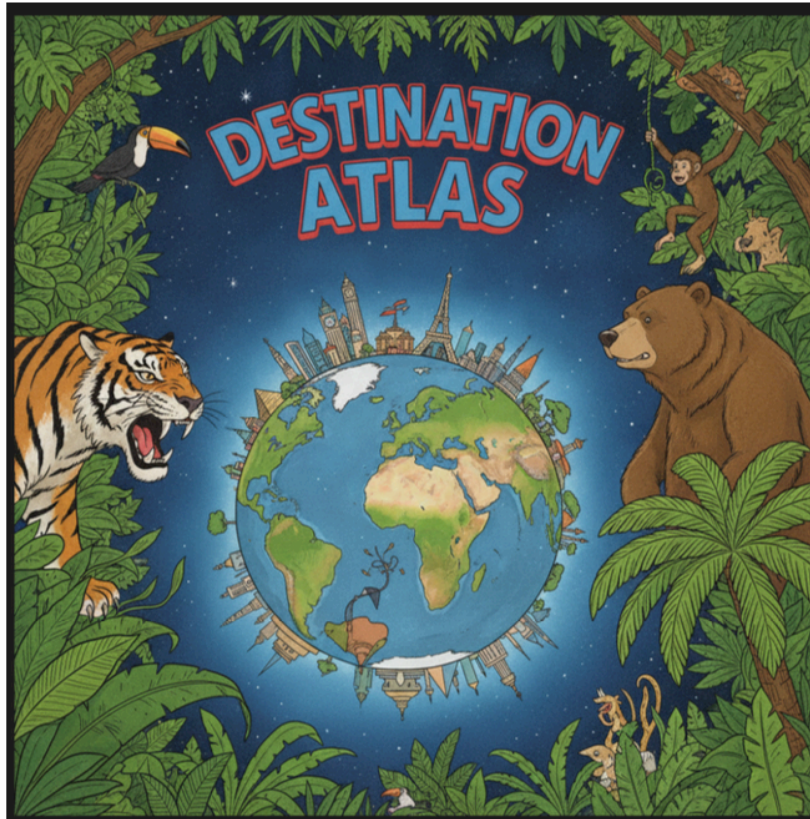


Figure 1: Destination Atlas awaits.

Five proof apps share one Destination Atlas library and the same screen names. This article uses the Angular proof:

```
npm run proof:angular
```

Open <http://localhost:4312>. The other runtimes follow the same screens.

The workbench

Every tab except About is a two-pane workbench. On the left is the Atlas preview. On the right is **Component source** — the template and class for that screen: imports, props, and nesting. On a narrow viewport, you toggle the panes rather than viewing them side by side.

About is the only page-level scroller. The shell locks the body. The other tabs are meant to fit the viewport.

The workbench layout appears again in the Maps and Settings articles.

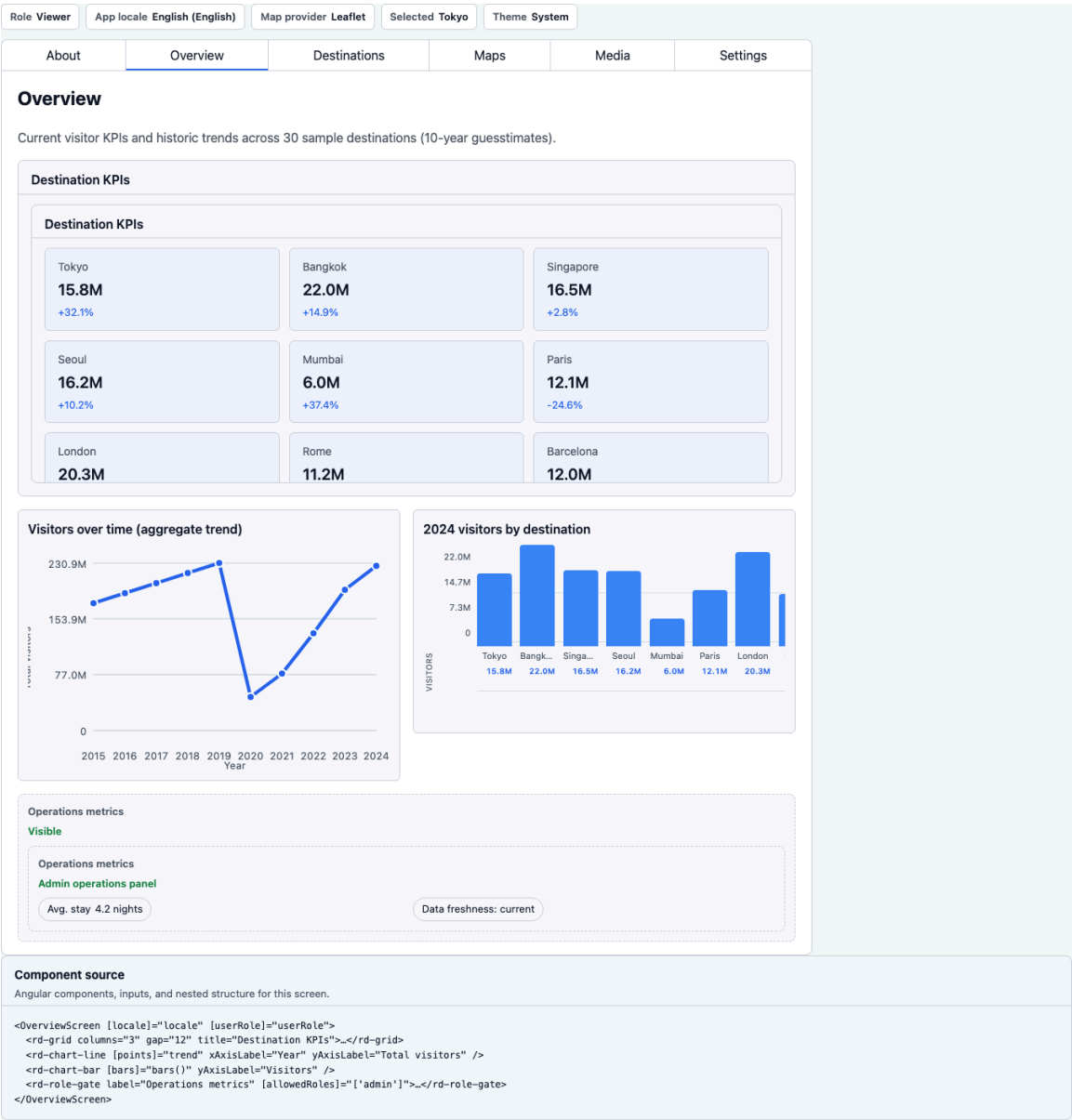


Figure 2: Workbench — preview on the left, component source on the right.

About, Overview, and Destinations

About. Why the demo exists, and a matrix of the five runtimes: package, proof command, and Storybook port. The current row is marked “You are here.”

Destination Atlas

Current and historic information about world locations — Angular proof (DAS-139)

Role Viewer

App locale English (English)

Map provider Leaflet

Selected Tokyo

Theme System

About

Overview

Destinations

Maps

Media

Settings

About Destination Atlas

Destination Atlas is a functional demo — not a component kitchen sink. Each screen exercises real RosettaDash components the way you would wire them in production: filters bound to tables, maps with provider choice, role gates, and media embeds.

Why Destination Atlas?

The five proof apps (Web Components + four frameworks) share a 30-city mock library from `libs/destination-atlas` and identical screen names. Compare runtime imports side-by-side, then open Storybook on the matching port to inspect components in isolation.

Install `@rosettadash/core` and the runtime package you need (`@rosettadash/react`, `/angular`, `/vue`, `/svelte`, or `/web-components`). See `docs/39-npm-consumer-install.md` for tarball and registry workflows.

Runtimes — proof apps & Storybook

Each runtime ships a **proof app** (full Destination Atlas UX) and a **Storybook catalog** (isolated component review). Use the same npm package in your consumer project.

The Web Components, React, Angular, Vue, and Svelte blocks below are not RosettaDash palette components — they are npm runtime packages (delivery targets). Each card is proof-app documentation UI: plain layout markup styled for this About page. The actual reusable components live inside those packages (KpiCard, GeoMap, ScrollRegion, etc.) and appear on the other tabs and in Storybook.

PACKAGE	PROOF APP	STORYBOOK
Web Components (Custom Elements) DAS-121 Shadow-DOM hosts and catalog elements. Map and Globe stay native <code>rd-*</code> custom elements; Authoring hosts the React subtree (DAS-159). Use when exporting CE-first dashboards or embedding <code>rd-*</code> tags in any stack. <code>@rosettadash/web-components</code> <code>npm run proof:web-components</code> localhost:4310 <code>npm run storybook:web-components</code> localhost:6006		

Figure 3: About — why the explorer exists, and the runtime matrix with “You are here.”

Overview. KPIs, a line chart, a bar chart, chips, badges, and a grid. This may be the dashboard shape most teams recognize.

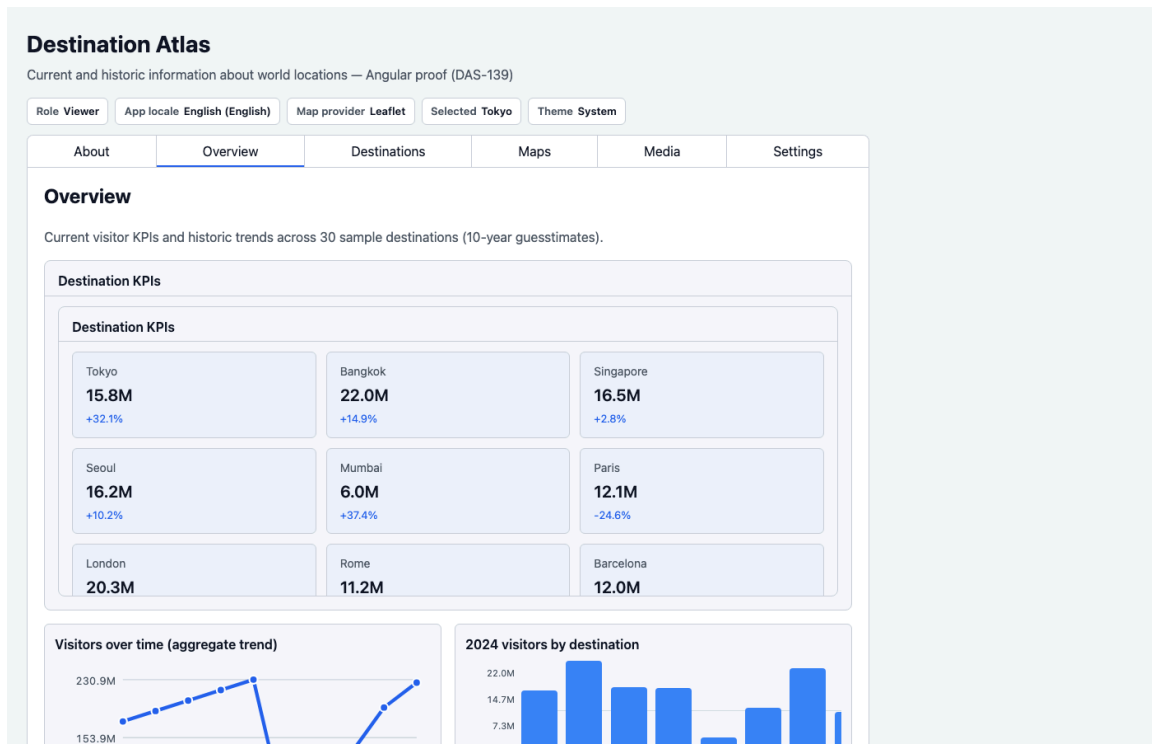


Figure 4: Overview — KPIs, line, bar. The dashboard everyone already knows how to ask for.

Destinations. Search, region filter, a date range, time presets, a table, and a detail panel. Filter to table to detail is the composite from the builder article, now backed by a thirty-city library. Selecting a row gives the rest of the app a selected place.

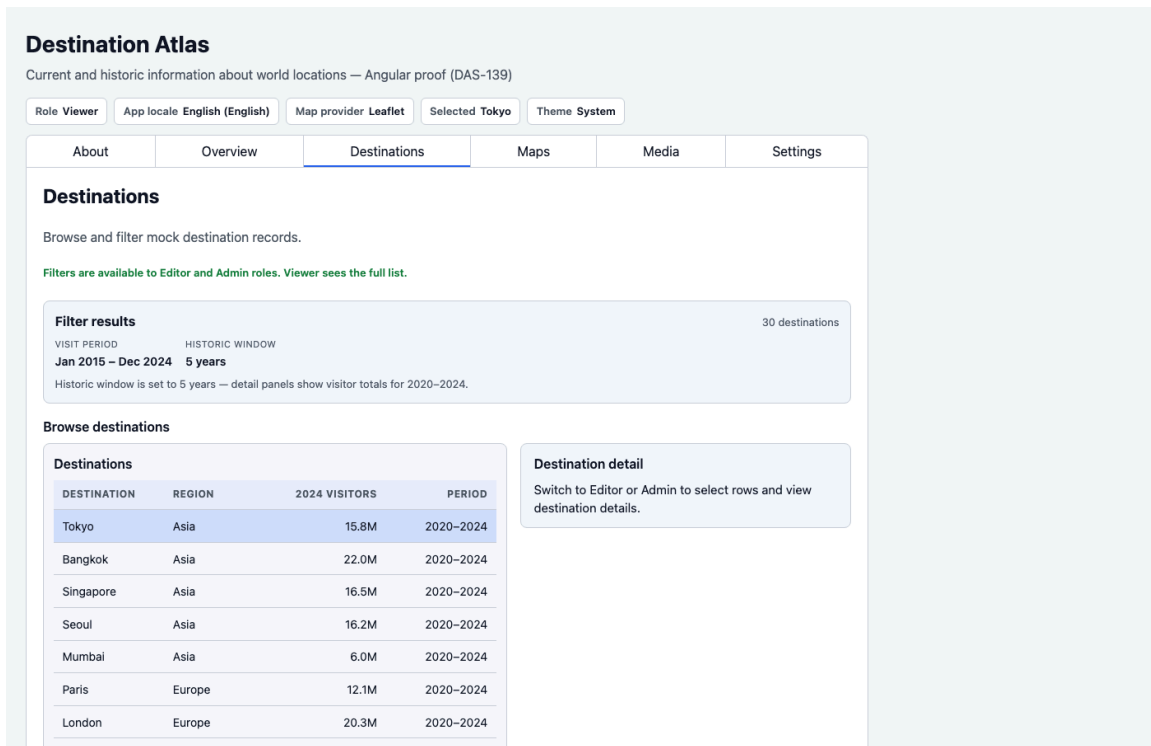


Figure 5: Destinations — filter to table to detail on the thirty-city library.

Those three screens are the focus of this article, on one runtime.

The remaining tabs

The other tabs are named here and covered in later articles. Important to remember is that these merely demonstrate functionality. You can use the code freely on your own projects as you please, which means taking over the look and feel.

Maps: presented in 2D maps and a 3D globe, data-bound to the current destination; **Media** is a simple YouTube player, playing videos of the current destination; **Authoring** lets you select 360 media and create a video from a rectangular portion of it; **Plan** (Admin) — for planning a trip, create details, team members, assign roles; **Stack** (Admin) shows export-wizard infra nodes. On the React and Angular proofs, when Docker is running, it also loads live seeded orders rows from a generated API — the walkthrough is in article 6. **Settings** — does what Settings usually does: it is the place where roles, BYOK, providers, and all the header states are set. **News** — destination-scoped travel headlines from Google News RSS via the builder API (~24h cache).

Optional graphic after Destinations: capture the News tab with a destination selected (graphics/05-news.png). React proof on :4311 is a good default.

Next

The next article deals with moving elements: 2D and 3D maps, 2D and 3D videos, and extracting media with JavaScript.